Record : DAA lab
Experiment no:

Rollno:160123749024

Date:

Experiment

Aim:

To perform minimum spanning tree using kruskals and prims algorithm

Description:

What is a Minimum Spanning Tree?

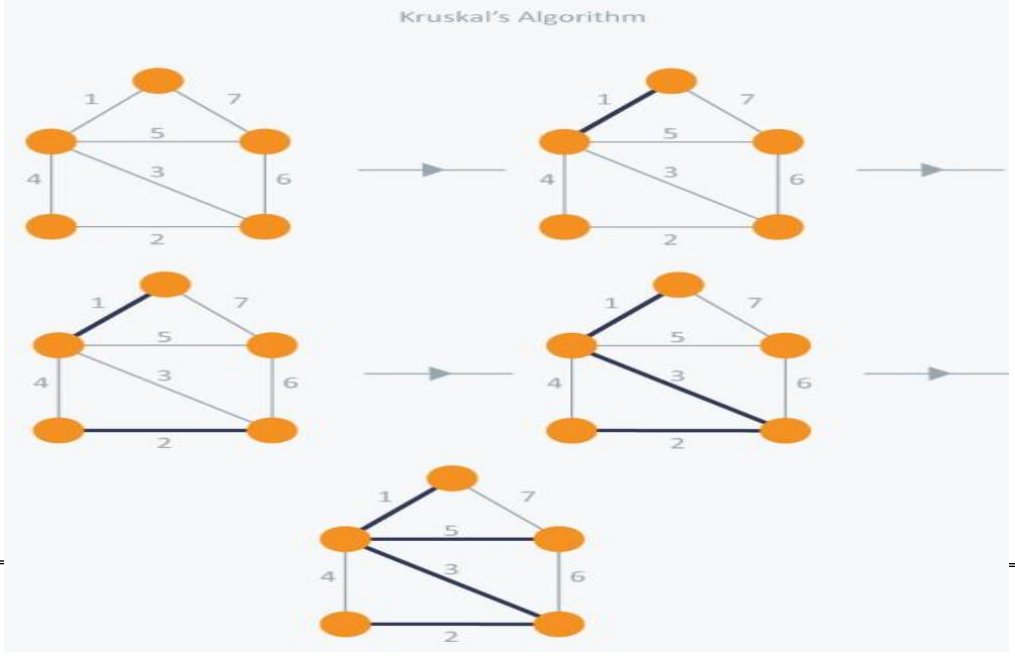
The cost of the spanning tree is the sum of the weights of all the edges in the tree. There can be many spanning trees. Minimum spanning tree is the spanning tree where the cost is minimum among all the spanning trees. There also can be many minimum spanning trees.

Minimum spanning tree has direct application in the design of networks. It is used in algorithms approximating the travelling salesman problem, multi-terminal minimum cut problem and minimum-cost weighted perfect matching.

Kruskal's Algorithm

Kruskal's Algorithm builds the spanning tree by adding edges one by one into a growing spanning tree. Kruskal's algorithm follows greedy approach as in each iteration it finds an edge which has least weight and add it to the growing spanning tree.

Example

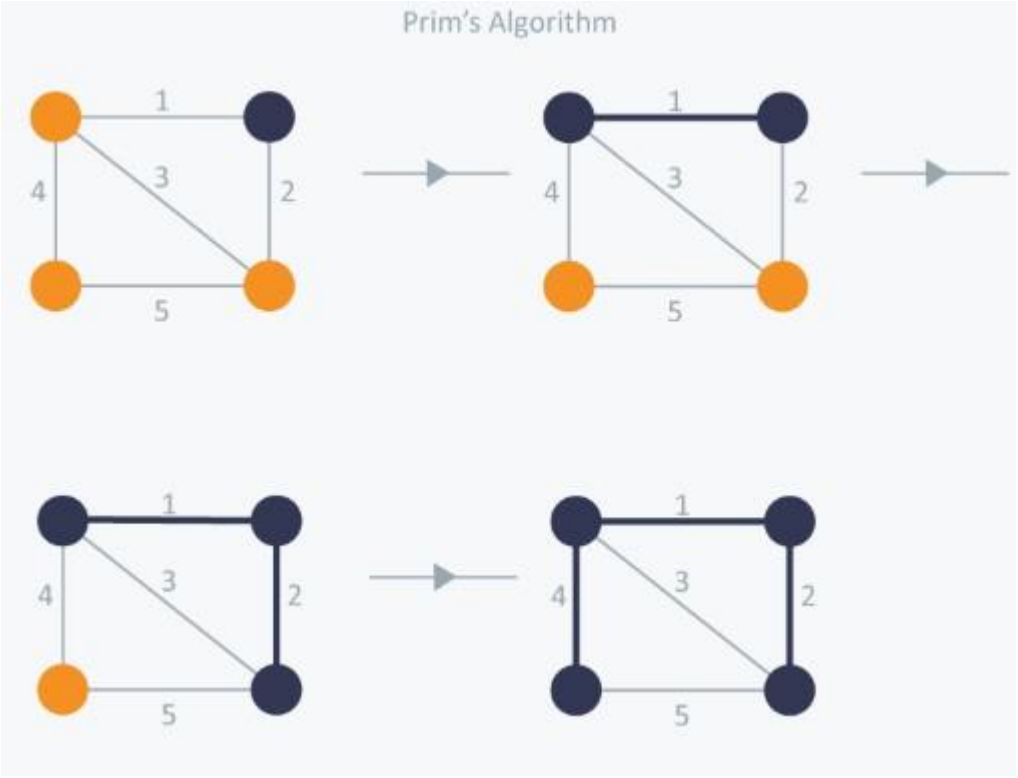


in Kruskal's algorithm, at each iteration we will select the edge with the lowest weight. So, we will start with the lowest weighted edge first i.e., the edges with weight 1. After that we will select the second lowest weighted edge i.e., edge with weight 2. Notice these two edges are totally disjoint. Now, the next edge will be the third lowest weighted edge i.e., edge with weight 3, which connects the two disjoint pieces of the graph. Now, we are not allowed to pick the edge with weight 4, that will create a cycle and we can't have any cycles. So we will select the fifth lowest weighted edge i.e., edge with weight 5. Now the other two edges will create cycles so we will ignore them. In the end, we end up with a minimum spanning tree with total cost 11 (= 1 + 2 + 3

+ 5).

Prim's Algorithm

Prim's Algorithm also use Greedy approach to find the minimum spanning tree. In Prim's Algorithm we grow the spanning tree from a starting position. Unlike an edge in Kruskal's, we add vertex to the growing spanning tree in Prim's.



In Prim's Algorithm, we will start with an arbitrary node (it doesn't matter which one) and mark it. In each iteration we will mark a new vertex that is adjacent to the one that we have already marked. As a greedy algorithm, Prim's algorithm will select the cheapest edge and mark the vertex. So we will simply choose the edge with weight 1. In the next iteration we have three

options, edges with weight 2, 3 and 4. So, we will select the edge with weight 2 and mark the vertex. Now again we have three options, edges with weight 3, 4 and 5. But we can't choose edge with weight 3 as it is creating a cycle. So we will select the edge with weight 4 and we end up with the minimum spanning tree of total cost 7 (= 1 + 2 +4).

Algorithm:

Kruskals algo

Sort all edges in **ascending order** based on their weights.

Initialize a **Disjoint Set Union (DSU)** structure to keep track of connected components. Iterate through the sorted edge list:

- For each edge (u, v, weight):
  - If `u` and `v` are in **different sets** (i.e., adding the edge won't form a cycle):
    - **Add the edge to the MST**.
    - **Union** the sets of `u` and `v`.
    - Add the weight to the **total MST cost**.

Stop when the MST has **V-1 edges** (where V = number of nodes). Return the **total minimum cost** and optionally the MST edges.

Prims algo

1. **Start** from any node (usually node `1`).
2. Create a **min-heap (priority queue)** and insert the starting node with weight `0`.
3. While the heap is not empty:

4.
   - Pop the node with the **minimum weight**.
   - If the node is already **visited**, skip it.
   - Otherwise:
     - **Add its weight to the total cost**.
     - Mark it as visited.
     - Push all **unvisited neighbors** of this node into the heap.
5. Repeat until all nodes are included in the MST.
6. Return the **total minimum cost**.

Code:

Kruskals algo

```
#kruskals
class DisjointSet:
    def __init__(self, n):
        self.parent = list(range(n + 1))

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])  # Path compression
        return self.parent[x]

    def union(self, x, y):
        root_x = self.find(x)
        root_y = self.find(y)
        self.parent[root_x] = root_y

def kruskal(nodes, edges, edge_list):
    dsu = DisjointSet(nodes)
    edge_list.sort()  # Sort edges by weight
    minimum_cost = 0
    mst_edges = []

    for weight, u, v in edge_list:
        if dsu.find(u) != dsu.find(v):
```

```
            minimum_cost += weight
            mst_edges.append((u, v, weight))
            dsu.union(u, v)

    return minimum_cost, mst_edges

# Main program
if __name__== "__main__":
    print(" Kruskal's Algorithm - Minimum Spanning Tree")
    print("-----------------------------------------------")
    print("Enter number of nodes and edges (e.g., '4 5'):")
    nodes, edges = map(int, input().split())

    edge_list = []
    print(f"\nNow enter {edges} edges in the format: node1 node2 weight")
    print("Example: 1 2 3 means an edge between node 1 and 2 with weight 3")

    for _ in range(edges):
        u, v, weight = map(int, input().split())
        edge_list.append((weight, u, v))

    print("\n Running Kruskal's Algorithm...")
    cost, mst = kruskal(nodes, edges, edge_list)

    print("\n Minimum Spanning Tree (MST) edges:")
    for u, v, w in mst:
        print(f"Edge between {u} and {v} with weight {w}")

    print(f"\n Total cost of the MST: {cost}")
```

Prims algo

```
#prims
import heapq

MAX = 10**4 + 5
marked = [False] * MAX
adj = [[] for _ in range(MAX)]

def prim(start):
    min_heap = []
    heapq.heappush(min_heap, (0, start))  # (weight, node)
    minimum_cost = 0

    while min_heap:
        weight, node = heapq.heappop(min_heap)
        if marked[node]:
```

```
continue

            marked[node] = True
        minimum_cost += weight

        for edge_weight, neighbor in adj[node]:
            if not marked[neighbor]:
                heapq.heappush(min_heap, (edge_weight, neighbor))

    return minimum_cost

# Input reading with prompts
if __name__== "__main__":
    print("Enter the number of nodes and edges (e.g., '4 5'):")
    nodes, edges = map(int, input().split())

    print(f"\nNow enter {edges} edges in the format: node1 node2 weight")
    print("Example: 1 2 3")
    for _ in range(edges):
        u, v, weight = map(int, input().split())
        adj[u].append((weight, v))
        adj[v].append((weight, u))

    print("\nCalculating Minimum Spanning Tree using Prim's Algorithm...")
    result = prim(1)  # Start from node 1
    print(f"\n The total minimum cost to connect all nodes is: {result}")
```

Output

```
 Kruskal's Algorithm - Minimum Spanning Tree
-----------------------------------------------
Enter number of nodes and edges (e.g., '4 5'):
4 5

Now enter 5 edges in the format: node1 node2 weight
Example: 1 2 3 means an edge between node 1 and 2 with weight 3
1 2 3
1 3 1
2 3 7
2 4 2
3 4 2

 Running Kruskal's Algorithm...

 Minimum Spanning Tree (MST) edges:
```

Edge between 1 and 3 with weight 1
Edge between 2 and 4 with weight 2
Edge between 1 and 2 with weight 3 Total cost of the

MST: 6

Enter the number of nodes and edges (e.g., '4 5'): 4 5
Now enter 5 edges in the format: node1 node2 weight

Example: 1 2 3
1 2 1
1 3 4
2 3 2
2 4 7
3 4 3

Calculating Minimum Spanning Tree using Prim's Algorithm... The total

minimum cost to connect all nodes is: 6